```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

import xgboost as xgb

from sklearn import metrics
```

```
insurance_dataset = pd.read_csv('/content/insurance.csv')
```

```
insurance_dataset.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Next steps: Generate code with `insurance_dataset` New interactive sheet

```
insurance_dataset.shape
```

```
(1338, 7)
```

```
insurance_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
insurance_dataset.isnull().sum()
```

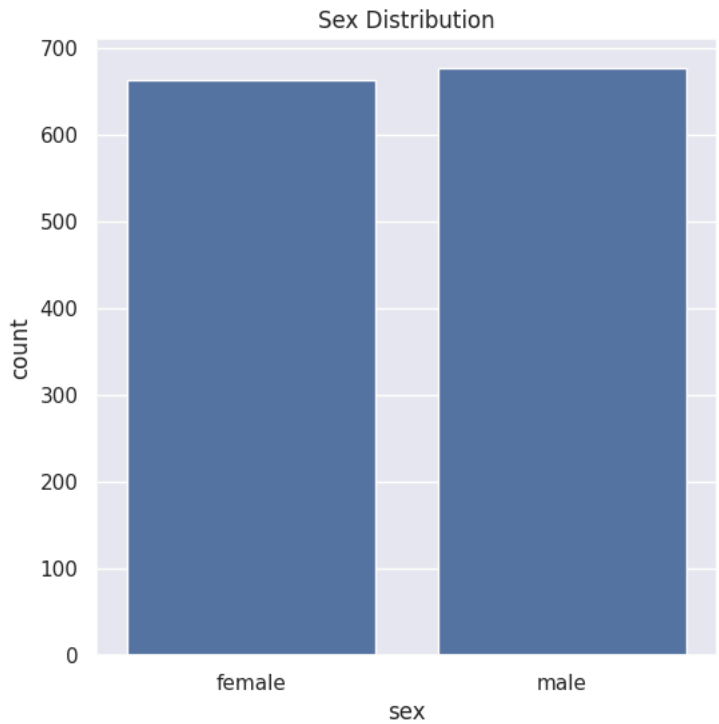
	0
age	0
sex	0
bmi	0
children	0
smoker	0
region	0
charges	0

dtype: int64

```
insurance_dataset.describe()
```

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

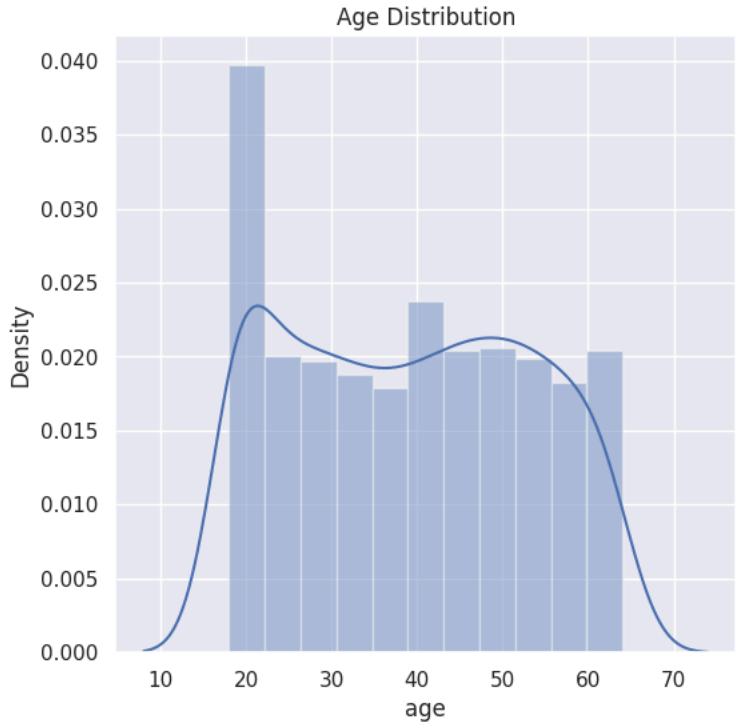
```
sns.set()
plt.figure(figsize=(6,6))
sns.countplot(x='sex', data=insurance_dataset)
plt.title('Sex Distribution')
plt.show()
```




```
sns.set()
plt.figure(figsize=(6,6))
sns.distplot(insurance_dataset['age'])
plt.title('Age Distribution')
plt.show()
```

```
/tmp/ipykernel_10614/1211896966.py:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```

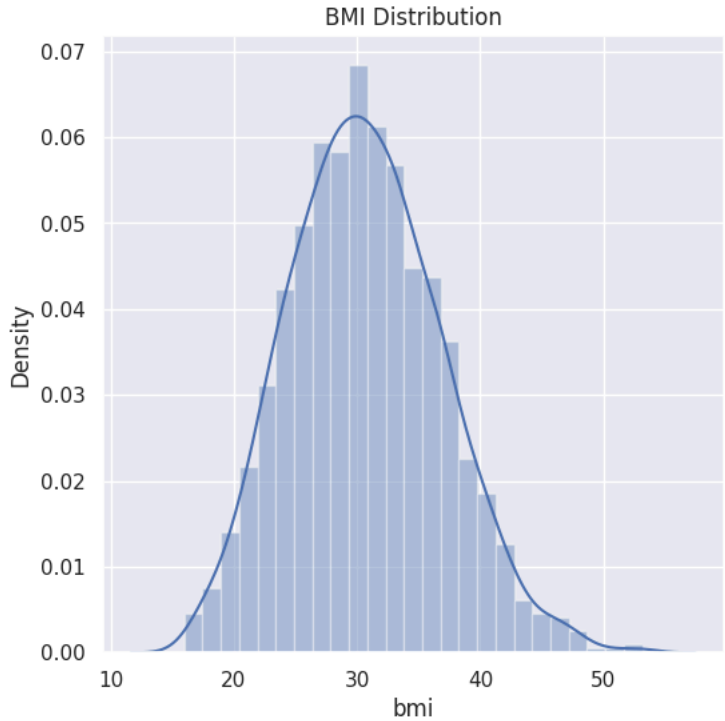
```
sns.distplot(insurance_dataset['age'])
```



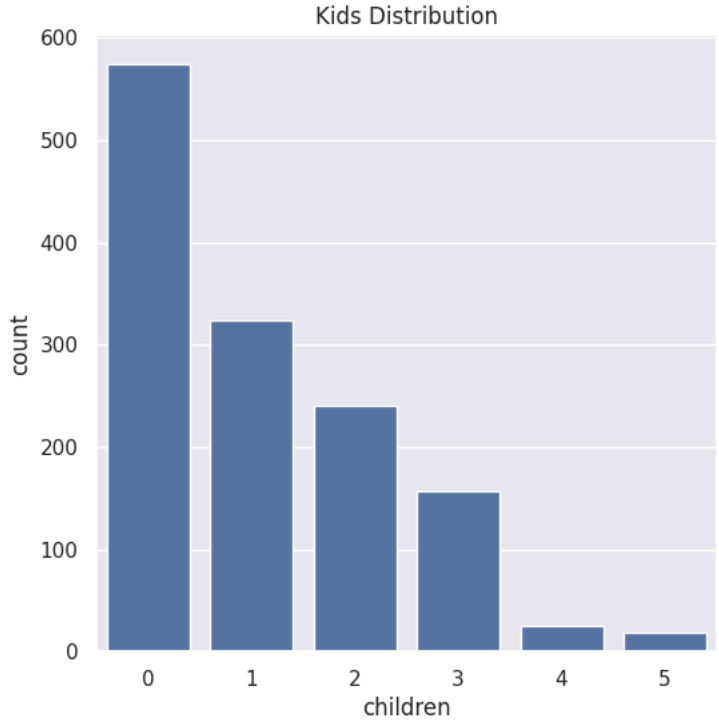

```
sns.set()
plt.figure(figsize=(6,6))
sns.distplot(insurance_dataset['bmi'])
plt.title('BMI Distribution')
plt.show()
```

```
/tmp/ipykernel_10614/1737976403.py:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```

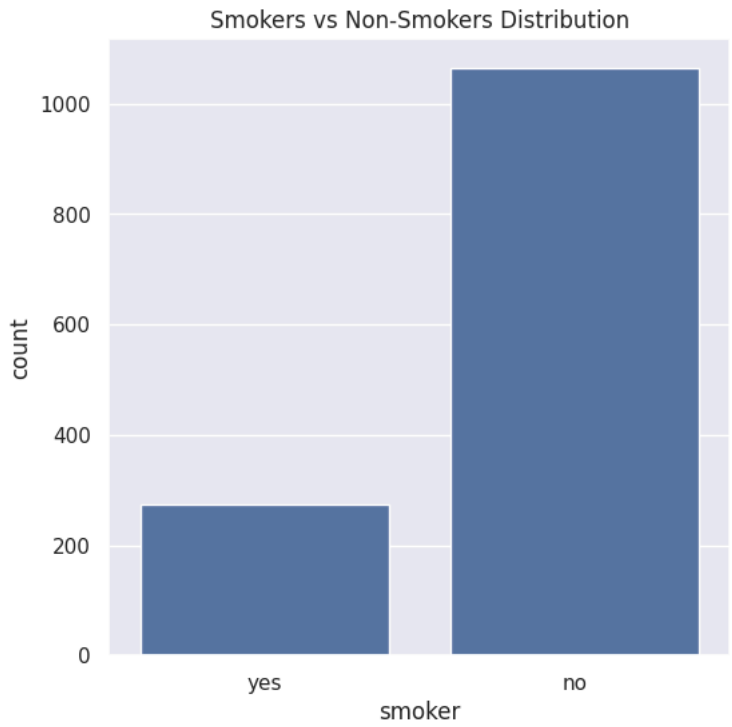
```
sns.distplot(insurance_dataset['bmi'])
```



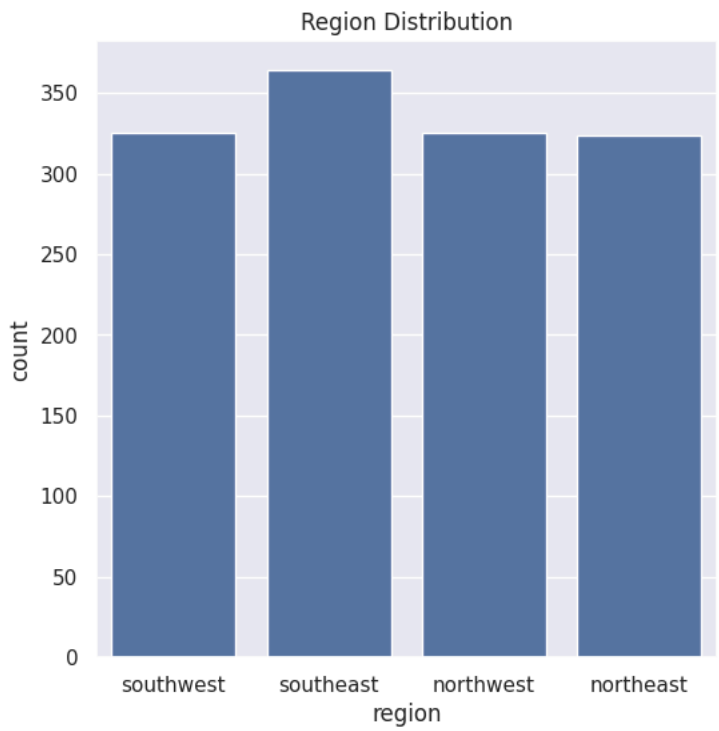

```
sns.set()
plt.figure(figsize=(6,6))
sns.countplot(x='children', data=insurance_dataset)
plt.title('Kids Distribution')
plt.show()
```



```
sns.set()
plt.figure(figsize=(6,6))
sns.countplot(x='smoker', data=insurance_dataset)
plt.title('Smokers vs Non-Smokers Distribution')
plt.show()
```



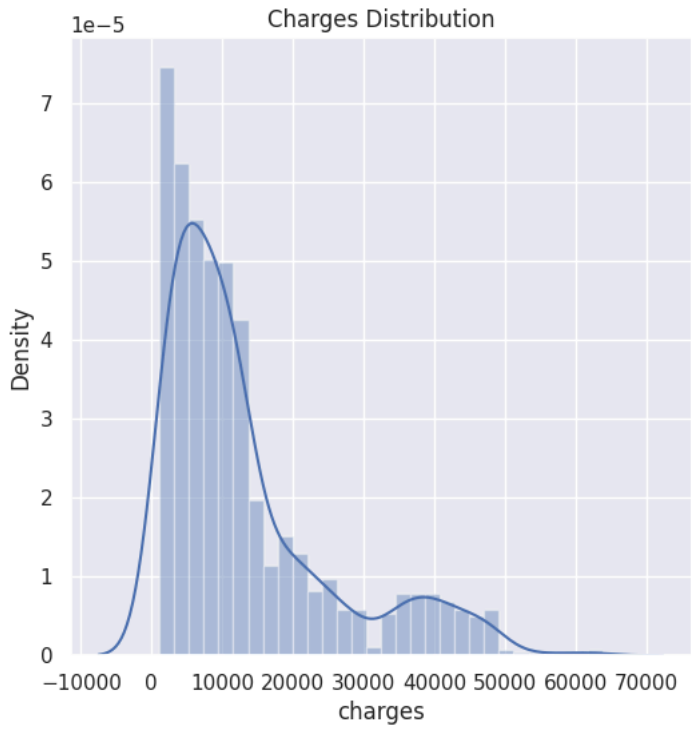

```
sns.set()
plt.figure(figsize=(6,6))
sns.countplot(x='region', data=insurance_dataset)
plt.title('Region Distribution')
plt.show()
```




```
sns.set()
plt.figure(figsize=(6,6))
```

```
sns.distplot(insurance_dataset['charges'])
plt.title('Charges Distribution')
plt.show()
```

```
/tmp/ipykernel_10614/3811374003.py:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(insurance_dataset['charges'])
```




```
insurance_dataset['region'].value_counts()
```

region	count
southeast	364
southwest	325
northwest	325
northeast	324

dtype: int64

```
insurance_dataset['sex'].value_counts()
```

sex	count
male	676
female	662

dtype: int64

```
insurance_dataset['smoker'].value_counts()
```

smoker	count
no	1064
yes	274

dtype: int64

```
insurance_dataset['children'].value_counts()
```

children	count
0	574
1	324
2	240
3	157
4	25
5	18

dtype: int64

```
insurance_dataset.replace({'male':0,'female':1}, inplace=True)
```

```
/tmp/ipykernel_10614/3299734873.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a fu
  insurance_dataset.replace({'male':0,'female':1}, inplace=True)
```

```
insurance_dataset.replace({'yes':0,'no':1}, inplace=True)
```

```
/tmp/ipykernel_10614/1976284366.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a fu
  insurance_dataset.replace({'yes':0,'no':1}, inplace=True)
```

```
insurance_dataset.replace({'southwest':0,'southeast':1,'northwest':2,'northeast':3}, inplace=True)
```

```
/tmp/ipykernel_10614/1289229596.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a fu
  insurance_dataset.replace({'southwest':0,'southeast':1,'northwest':2,'northeast':3}, inplace=True)
```

```
insurance_dataset.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	1	27.900	0	0	0	16884.92400
1	18	0	33.770	1	1	1	1725.55230
2	28	0	33.000	3	1	1	4449.46200
3	33	0	22.705	0	1	2	21984.47061
4	32	0	28.880	0	1	2	3866.85520

Next steps: Generate code with `insurance_dataset` New interactive sheet

```
insurance_dataset.to_csv('insurance_dataset_preprocessed.csv')
```

```
X = insurance_dataset.drop(columns='charges', axis=1)
Y = insurance_dataset['charges']
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(1338, 6) (1070, 6) (268, 6)
```

```
model = LinearRegression()
```

```
model.fit(X_train, Y_train)
```

```
LinearRegression
LinearRegression()
```

```
X_train_prediction = model.predict(X_train)
```

```
r2_train = metrics.r2_score(Y_train, X_train_prediction)
mae_train = metrics.mean_absolute_error(Y_train, X_train_prediction)
mse_train = metrics.mean_squared_error(Y_train, X_train_prediction)
rmse_train = np.sqrt(mse_train)
print("R Squared Train:", r2_train)
print("Mean Absolute Error Train:", mae_train)
print("Mean Squared Error Train:", mse_train)
print("Root Mean Squared Error Train:", rmse_train)
```

```
R Squared Train: 0.7519923667088932
Mean Absolute Error Train: 4140.028379854144
Mean Squared Error Train: 36104122.875132434
Root Mean Squared Error Train: 6008.670641259382
```

```
X_test_prediction = model.predict(X_test)
```

```
r2_test = metrics.r2_score(Y_test, X_test_prediction)
mae_test = metrics.mean_absolute_error(Y_test, X_test_prediction)
mse_test = metrics.mean_squared_error(Y_test, X_test_prediction)
rmse_test = np.sqrt(mse_test)
print("R Squared Test:", r2_test)
print("Mean Absolute Error Test:", mae_test)
print("Mean Squared Error Test:", mse_test)
print("Root Mean Squared Error Test:", rmse_test)
```

```
R Squared Test: 0.7445422986536503
Mean Absolute Error Test: 4285.219437633699
Mean Squared Error Test: 38364832.19054552
Root Mean Squared Error Test: 6193.935113523997
```

```
X_train_accuracy = model.score(X_train, Y_train)
X_test_accuracy = model.score(X_test, Y_test)
print("Train Accuracy:", X_train_accuracy)
print("Test Accuracy:", X_test_accuracy)
```

```
Train Accuracy: 0.7519923667088932
Test Accuracy: 0.7445422986536503
```

```
xgb_model = xgb.XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    objective='reg:squarederror' )
```

```
xgb_model.fit(X_train, Y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.1, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=5,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=100,
             n_jobs=None, num_parallel_tree=None, ...)
```

```
XGB_train_prediction = xgb_model.predict(X_train)
```

```
r2_xgb_train = metrics.r2_score(Y_train, XGB_train_prediction)
mae_xgb_train = metrics.mean_absolute_error(Y_train, XGB_train_prediction)
mse_xgb_train = metrics.mean_squared_error(Y_train, XGB_train_prediction)
rmse_xgb_train = np.sqrt(mse_xgb_train)
print("R Squared XGB Train:", r2_xgb_train)
print("Mean Absolute Error XGB Train:", mae_xgb_train)
print("Mean Squared Error XGB Train:", mse_xgb_train)
print("Root Mean Squared Error XGB Train:", rmse_xgb_train)
```

```
R Squared XGB Train: 0.9434101109966285
Mean Absolute Error XGB Train: 1539.756098437405
Mean Squared Error XGB Train: 8238167.023148214
Root Mean Squared Error XGB Train: 2870.2207272522114
```

```
XGB_test_prediction = xgb_model.predict(X_test)
```

```
r2_xgb_test = metrics.r2_score(Y_test, XGB_test_prediction)
mae_xgb_test = metrics.mean_absolute_error(Y_test, XGB_test_prediction)
mse_xgb_test = metrics.mean_squared_error(Y_test, XGB_test_prediction)
rmse_xgb_test = np.sqrt(mse_xgb_test)
print("R Squared XGB Test:", r2_xgb_test)
print("Mean Absolute Error XGB Test:", mae_xgb_test)
print("Mean Squared Error XGB Test:", mse_xgb_test)
print("Root Mean Squared Error XGB Test:", rmse_xgb_test)
```

```
R Squared XGB Test: 0.8572735666934079
Mean Absolute Error XGB Test: 2530.9088776444437
Mean Squared Error XGB Test: 21434764.480005115
Root Mean Squared Error XGB Test: 4629.769376546214
```

```
XGB_test_accuracy = xgb_model.score(X_test, Y_test)
print("Test Accuracy XGB:", XGB_test_accuracy)

XGB_train_accuracy = xgb_model.score(X_train, Y_train)
print("Train Accuracy XGB:", XGB_train_accuracy)
```

```
Test Accuracy XGB: 0.8572735666934079
Train Accuracy XGB: 0.9434101109966285
```